Game Development Patterns

Get ready for Yash's very indulgent talk about game programming!!! AAAAAAHHHH

Prelude

What is this presentation? Who is it for?

This is a programming focused presentation for the following:

- Know the basics of C# or C++ at this point
- Keep running into spaghetti code unstable solutions to problems, and weird workarounds
- Abandoning projects out of frustration.

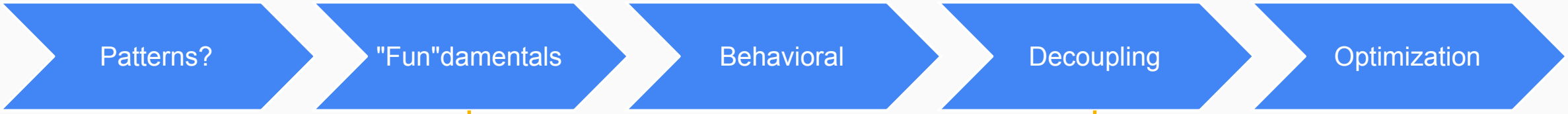
If any of these sound like you, this is your presentation.

Here's the Business

Introduce you to what patterns are, why we use them, what they are for

Patterns for telling the things in your game what to do

Patterns for making the game run good



Basic patterns and how to use them

Patterns for making your code easier to modify on a small scale

What is a game development pattern??

I'll give you three answers, take the one you like.

- A solution to problems we commonly run into.
- More accurately, patterns are tools
- Something to aid your brain when problem solving.

Why use a pattern?

There's a lot of reasons, but for the sake of time

- Organizes code
- Easier for others to understand
- Optimizes your solutions*
- Good software architecture

What would I use a pattern for exactly?

All kinds of things.

- Collision Detection
- Optimal Rendering
- AI Decision Making
- Managing Objects

Just to name a few.

That didn't help, Yash I feel overwhelmed

I'm legitimately sorry.

Fundamental Patterns

Command

Singleton

State Machines

The gang of four described the Command pattern as follows

"Encapsulate a request as an object, thereby letting users parameterize clients with different requests, queue or log requests, and support undoable operations."

I'll tell you what this is, it's an awful definition. Maybe they just were having an off day, let's give the gang another chance.

The gang of four later describes:

"Commands are an object-oriented replacement for callbacks."

Better. However, here's an even more concise definition.

"A command is a method call made into an object"

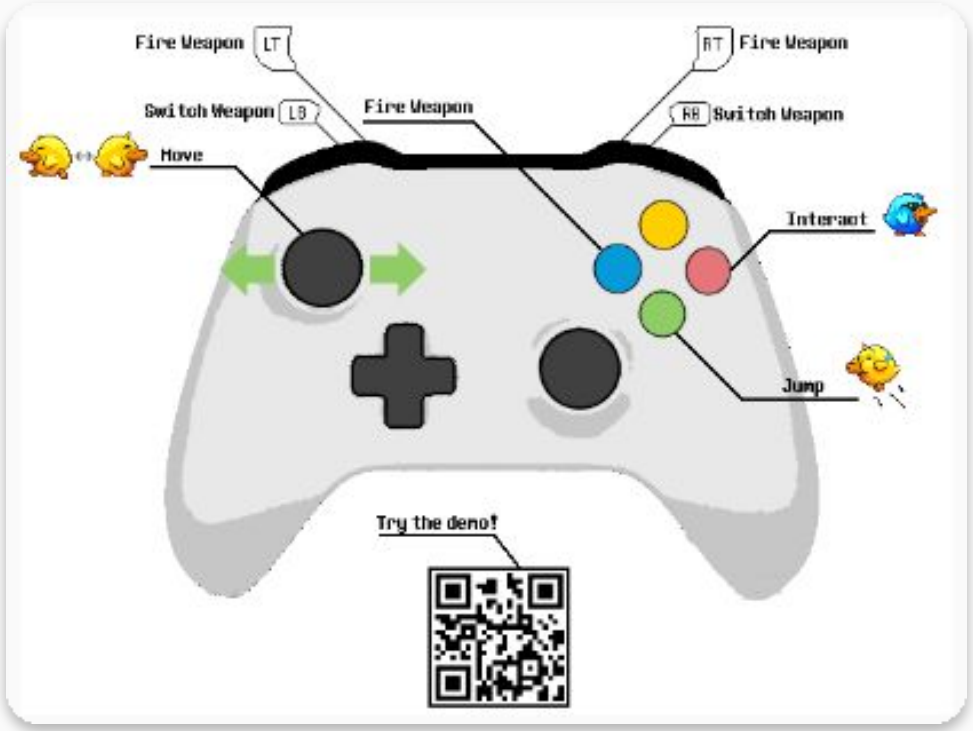
Okay Yash, how is that any different than a function pointer or callback, what have you?

I know it sounds like a function pointer; it's similar, like you're in the same neighborhood, but it's not I promise.

Let's see an example.

Handling Input

Somewhere in a game is a chunk of code that reads user input - keyboard presses, mouse clicks, whatever. It takes each input and translates it to a meaningful action in the game



An easy implementation could look something like this:

```
void InputHandler::handleInput()
{
  if (isPressed(BUTTON_X)) player.jump();
  else if (isPressed(BUTTON_Y)) player.fireGun();
  else if (isPressed(BUTTON_A)) player.swapWeapon();
  else if (isPressed(BUTTON_B)) player.lurchIneffectively();
}
```

All code is in C++ btw, sorry C# gamers

What if I want rebindable controls?

- Turn direct calls into something swappable
- Object to represent action
- In other words, the command pattern

First let's make a base Command class:

```
class Command
{
public:
  virtual ~Command() {}
  virtual void execute(GameObject& obj) = 0;
};
```

Then create child classes for each action

```
class JumpCommand : public Command
{
public:
  virtual void execute(GameObject& obj)
  {
      obj.jump();
  }
};

class FireCommand : public Command
{
public:
  virtual void execute(GameObject& obj)
  {
      obj.fireGun();
  }
};

// You get the idea...
```

In our input handler, we store a pointer to a command for each button:

```
class InputHandler
{
public:
  void handleInput();

  // Methods to bind commands...

private:
  Command* buttonX_;
  Command* buttonY_;
  Command* buttonA_;
  Command* buttonB_;
};
```

Now the input handling gets delegated to those and we can return a command:

```
Command* InputHandler::handleInput()
{
  if (isPressed(BUTTON_X)) return buttonX_;
  if (isPressed(BUTTON_Y)) return buttonY_;
  if (isPressed(BUTTON_A)) return buttonA_;
  if (isPressed(BUTTON_B)) return buttonB_;

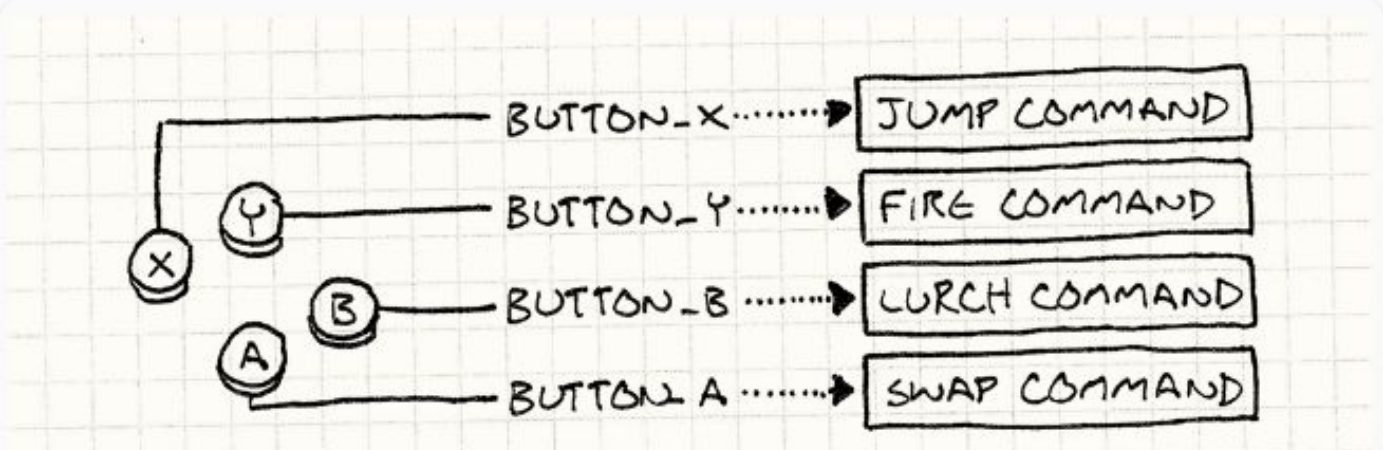
  // Nothing pressed, so do nothing.
  return NULL;
}
```

Then we add some code and our player or any other object we pass in, can be controlled:

```
Command* command = inputHandler.handleInput();
if (command)
{
  command->execute(player);
}
```

Benefit of the Command pattern

- Creates a layer of abstraction between object and the input
- Make it easier to change the methods going between



The Singleton

This is the most essential pattern to know for game development.

The gang of four defines the Singleton as:

"Ensure a class has one instance, and provide a global point of access to it."

Let's really try and understand this.

Breaking It Down

"Ensure a class has one instance"

Some classes cannot perform correctly with more than one instance

- External system that maintains its own state
- Game Manager that transitions between scenes
- Audio System that plays audio throughout the game

Breaking It Down

"provide a global point of access to it"

Several different systems will need to access our singleton, if there is only one instance, it needs to be global for all to access it.

- Needs to be able to act asynchronously
- No interference across different access points
- Singleton is the solution to all this

File System

A File System is a classic example of a singleton

- External system that keeps track of its own state
- Many of our systems will need to access.

An implementation could look something like this:

```
class FileSystem
{
public:
  static FileSystem& instance()
  {
    static FileSystem *instance = new FileSystem();
    return *instance;
  }

private:
  FileSystem() {}
};
```

```
public class FileSystem : MonoBehaviour
{

    public static FileSystem Instance;
    void Awake()
    {
        Instance = this;
    }
};
```

C++ and C# have two different ways of making a singleton

Why use this?

- It doesn't create an instance if no part of the program uses it.
- It's initialized at runtime. (lazy initialization)
- You can subclass the singleton

Making child classes:

```
// assuming we add a readFile and writeFile method to our base class
class PS4FileSystem : public FileSystem
{
public:
  virtual char* readFile(char* path)
  {
    // Use Sony file IO API...
  }

  virtual void writeFile(char* path, char* contents)
  {
    // Use sony file IO API...
  }
};

class SwitchFileSystem : public FileSystem
{
public:
  virtual char* readFile(char* path)
  {
    // Use Nintendo file IO API...
  }

  virtual void writeFile(char* path, char* contents)
  {
    // Use Nintendo file IO API...
  }
};
```

So now we have accommodated our singleton for the different use cases

The clever part:

```
FileSystem& FileSystem::instance()
{
  #if PLATFORM == PLAYSTATION4
    static FileSystem *instance = new PS4FileSystem();
  #elif PLATFORM == NSWITCH
    static FileSystem *instance = new SwitchFileSystem();
  #endif

  return *instance;
}
```

States

What's wrong with this behavior?

```
void Player::handleInput( Input input )
{
      if( input == BUTTON_B )
      {
            velocityY = jumpStrength;
            setAnimation( player_jump );
      }

}
```

So we add a flag....

```
void Player::handleInput( Input input )
{
      if( input === BUTTON_B )
      {
          if(!isJumping)
          {
              isJumping = true;
              //jumping code...
          }
      }
}
```

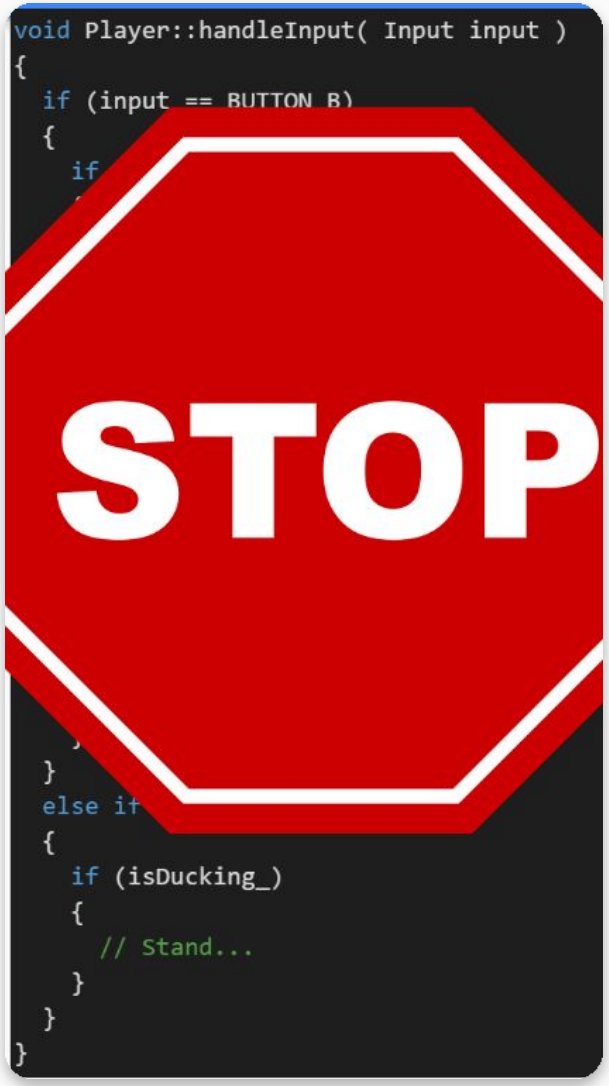
Now let's add crouching

```
void Player::handleInput( Input input )
{
  if (input == BUTTON_B)
  {
    // Jump if not jumping...
  }
  else if (input == BUTTON_DOWN)
  {
    if (!isJumping_)
    {
      setAnimation(IMAGE_DUCK);
    }
  }
  else if (input == RELEASE_DOWN)
  {
    setAnimation(IMAGE_STAND);
  }
}
```

See the bug this time?

So we add diving....

And there's another bug!
We check if you jump in the jump state, but not in the dive state... time to add another flag.

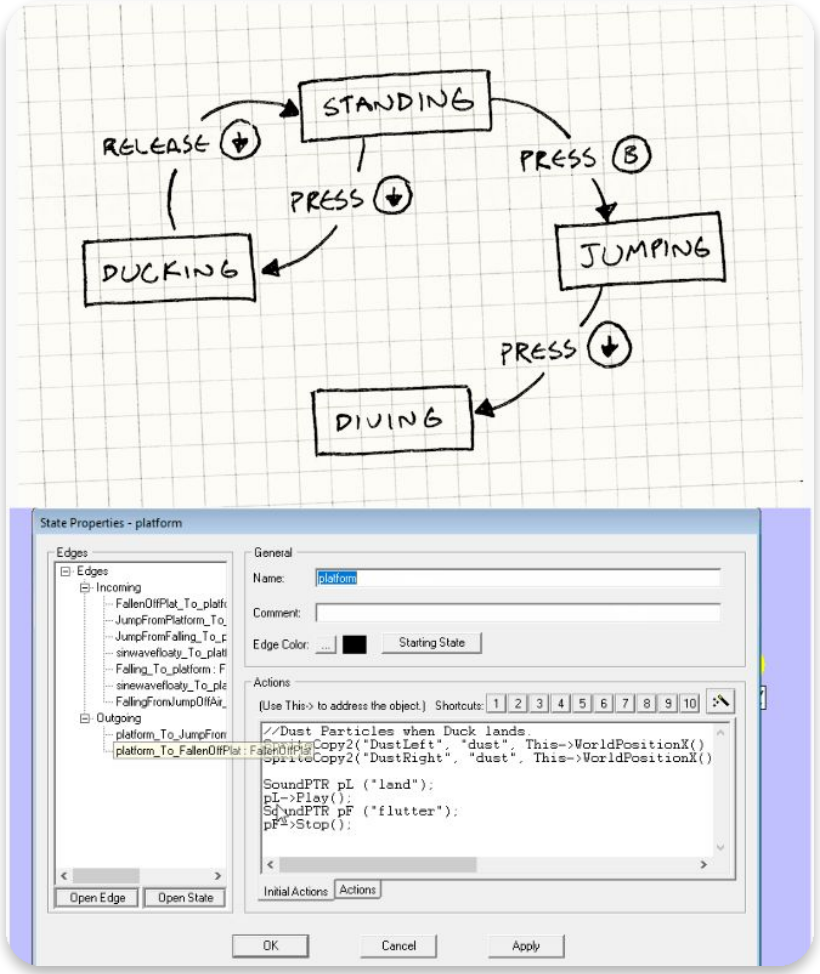


Finite State Machines

Let's document our states in a graph like so. We know what states we want and we know how we want to get from one state to the next.

Congrats! We've made a Finite State Machine.

- We have fixed states the machine (player) can be in
- The machine can only be in one state at a time.
- A sequence of inputs or events is sent
- Each state has a set of exit and enter conditions with a start and ending state.



Example state machine for a platformer's player controller

How do you implement that?

Many ways

- Enums and switches
- State Pattern / Dynamic dispatch (virtual method calls in C++)
- Static States

We can even abandon FSMs altogether

- Concurrent State Machines
  - When one is not enough
- Hierarchical State Machines
  - Get rid of arbitrary code
- Pushdown Automata
  - Keep history of states

Enum represents our states

```
enum State
{
  STATE_STANDING,
  STATE_JUMPING,
  STATE_DUCKING,
  STATE_DIVING
};
```

Switch statement machine

```
void Player::handleInput( Input input )
{
  switch (state_)
  {
    case STATE_STANDING:
      if (input == BUTTON_B)
      {
        state_ = STATE_JUMPING;
        //jump code
      }
      else if (input == BUTTON_DOWN)
      {
        state_ = STATE_DUCKING;
        setAnimation(IMAGE_DUCK);
      }
      break;
```

```
    case STATE_JUMPING:
      if (input == BUTTON_DOWN)
      {
        state_ = STATE_DIVING;
        setAnimation(IMAGE_DIVE);
      }
      break;

    case STATE_DUCKING:
      if (input == RELEASE_DOWN)
      {
        state_ = STATE_STANDING;
        setAnimation(IMAGE_STAND);
      }
      break;
  }
```

Glad that's over with!!



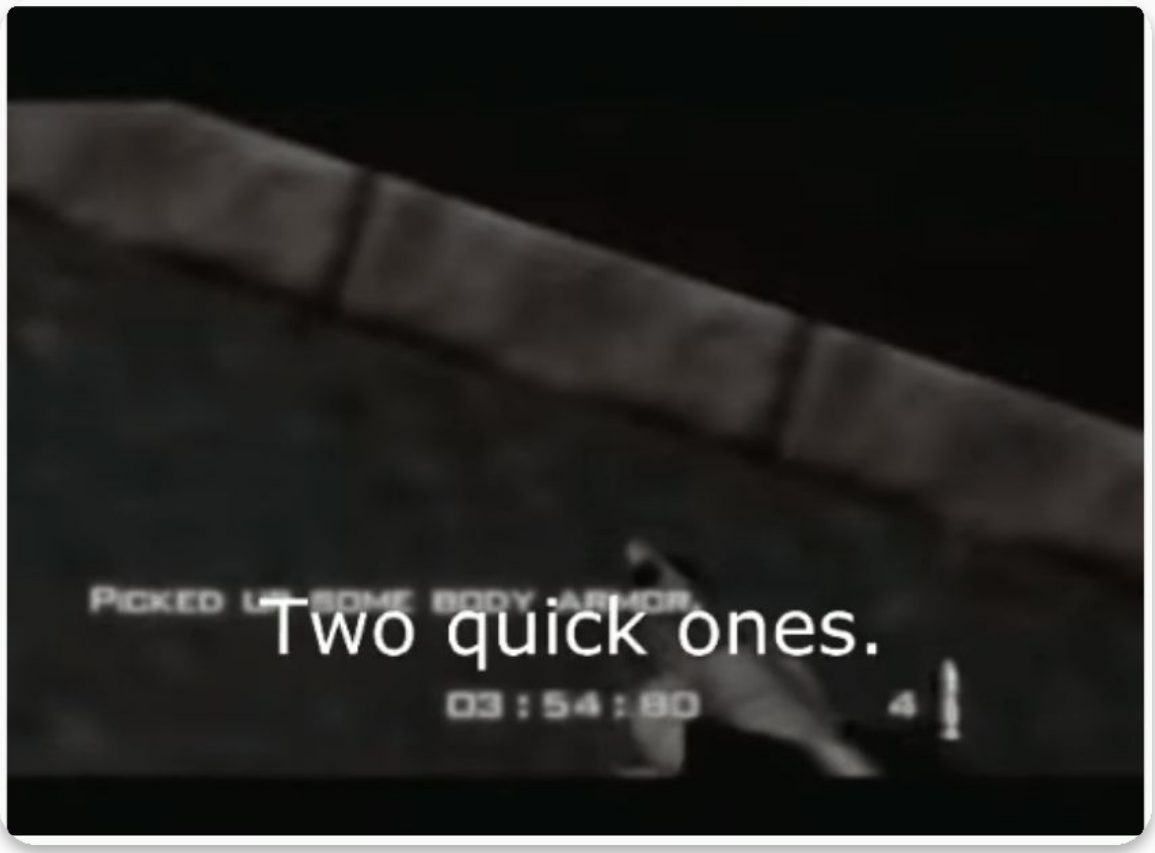


Image Credit: [Ryan Lockwood - Streets Agent 1:12]

Behavioral Patterns

Type Object

Subclass Sandbox

Type Object

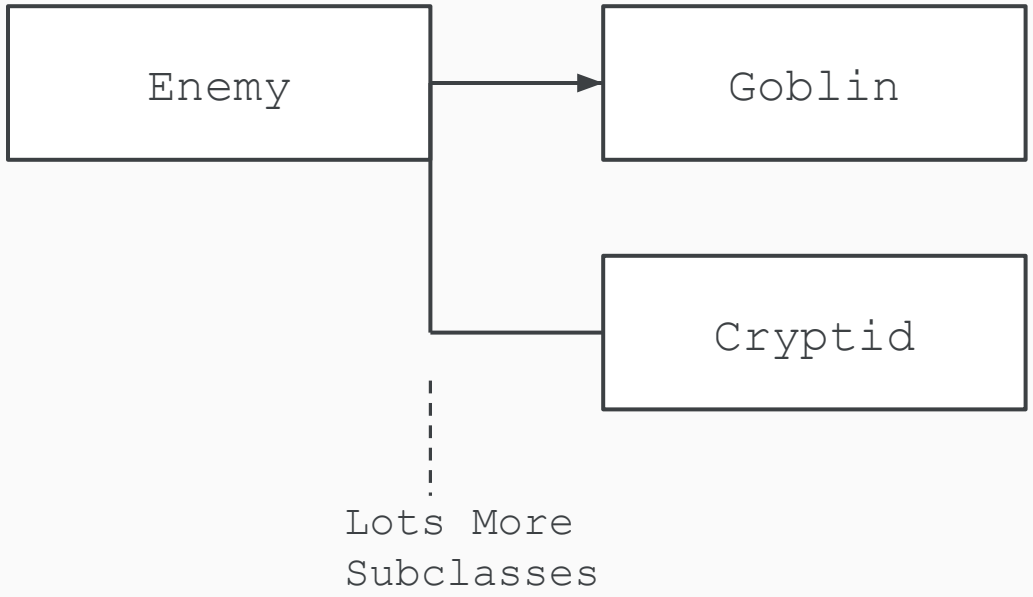
Design Problem: We have many enemies in a game. They all have strings, images, stats associated with them. How are we to implement all the enemies?



Image Credit: EarthBound / Mother 2

Typical Object Oriented Approach

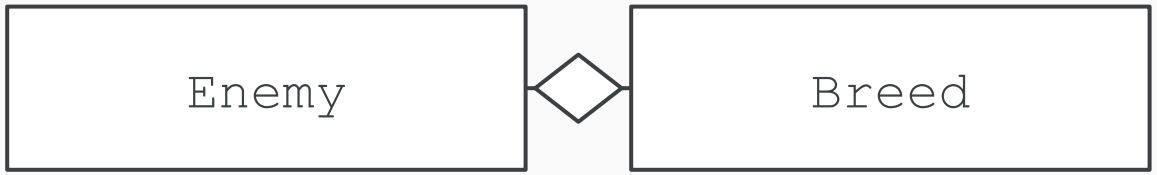
Let's just make a parent class and have everyone inherit from it!



Incorrect.

Type Object Approach

We just have two associated classes, Breed storing data for Enemy.



- All enemies are just enemy objects that look at an associated Breed object for their unique data
- Note that there is no inheritance here.
- Make new types of enemies without complicating the codebase at all.

Things to Be Aware Of

- In C++ especially, the type objects have to be tracked manually
  - Make sure breed is correctly referenced when it's needed
- It's harder to define behavior for each type
  - If you need a large range of defined behaviors for different enemies, don't use this pattern
- A fixed set of predefined behaviors is the limiting factor for different behavior.

Implementation

```
class Breed
{
public:
  Breed(int health, const char* attack)
  : health_(health),
    attack_(attack)
  {}

  int getHealth() { return health_; }
  const char* getAttack() { return attack_; }

private:
  int health_; // Starting health.
  const char* attack_;
};
```

```
class Enemy
{
public:
  Enemy(Breed& breed)
  : health_(breed.getHealth()),
    breed_(breed)
  {}

  const char* getAttack()
  {
    return breed_.getAttack();
  }

private:
  int    health_; // Current health.
  Breed& breed_;
};
```

Subclass Sandbox

But what do you do when you need a lot of classes with similar base functions but entirely branching behaviors? Use this pattern, the Subclass Sandbox, when:

- The base class is able to provide all of the operations that a child class may need to perform.
- There is behavioral overlap in the subclasses and you want to make it easier to share code between them.
- You want to minimize dependencies between those child classes and the rest of the program.

Yash, this sounds like just having a well structured parent class.

It's a bit different in functionality, let's look at an example.

Superpowers

```
class Superpower
{
public:
  virtual ~Superpower() {}

protected:
  virtual void activate() = 0;

  void move(double x, double y, double z)
  {
    // Code here...
  }

  void playSound(SoundId sound, double volume)
  {
    // Code here...
  }

  void spawnParticles(ParticleType type, int count)
  {
    // Code here...
  }
};
```

```
class SkyLaunch : public Superpower
{
protected:
  virtual void activate()
  {
    // Spring into the air.
    playSound(SOUND_SPROING, 1.0f);
    spawnParticles(PARTICLE_DUST, 10);
    move(0, 0, 20);
  }
};
```

```
class Superpower
{
protected:
  double getHeroX()
  {
    // Code here...
  }
  double getHeroY(){}
  // Existing stuff...
};
```

```
class SkyLaunch : public Superpower
{
protected:
  virtual void activate()
  {
    if (getHeroY() == 0)
    {
      // On the ground, so spring into the air.
      playSound(SOUND_SPROING, 1.0f);
      spawnParticles(PARTICLE_DUST, 10);
      move(0, 0, 20);
    }
    else if (getHeroY() < 10.0f)
    {
      // Near the ground, so do a double jump.
      playSound(SOUND_SWOOP, 1.0f);
      move(0, 0, getHeroZ() - 20);
    }
    else
    {
      // Way up in the air, so do a dive attack.
      playSound(SOUND_DIVE, 0.7f);
      spawnParticles(PARTICLE_SPARKLES, 1);
      move(0, 0, -getHeroZ());
    }
  }
```

Decoupling Patterns

Component

Service Locating

Component

Have you ever used Unity? Unreal? It's that. No further explanation needed.

Jokes aside, the component pattern is pretty much exactly how Unity uses it. Put behavioral objects, scripts, and patterns into "add-on parts" so that when one thing breaks it doesn't entangle all the other parts.

Quick Implementation

```
class InputComponent
{
public:
  void update(Player& plyr)
  {
    switch (Controller::getJoystickDirection())
    {
      case DIR_LEFT:
        plyr.velocity -= WALK_ACCELERATION;
        break;

      case DIR_RIGHT:
        plyr.velocity += WALK_ACCELERATION;
        break;
    }
  }

private:
  static const int WALK_ACCELERATION = 1;
};
```

```
class PhysicsComponent
{
public:
  void update(Player& plyr, World& world)
  {
    plyr.x += plyr.velocity;
    world.resolveCollision(volume_,
        plyr.x, plyr.y, plyr.velocity);
  }

private:
  Volume volume_;
};
```

```
class GraphicsComponent
{
public:
  void update(Player& plyr, Graphics& graphics)
  {
    Sprite* sprite = &spriteStand_;
    if (plyr.velocity < 0)
    {
      sprite = &spriteWalkLeft_;
    }
    else if (plyr.velocity > 0)
    {
      sprite = &spriteWalkRight_;
    }

    graphics.draw(*sprite, plyr.x, plyr.y);
  }

private:
  Sprite spriteStand_;
  Sprite spriteWalkLeft_;
  Sprite spriteWalkRight_;
};
```

```
class Player
{
public:
  int velocity;
  int x, y;

  void update(World& world, Graphics& graphics)
  {
    input_.update(*this);
    physics_.update(*this, world);
    graphics_.update(*this, graphics);
  }

private:
  InputComponent input_;
  PhysicsComponent physics_;
  GraphicsComponent graphics_;
};
```

Event Queue

This is gonna be a rough one. An event queue is defined as:

"Stores a series of requests in first-in, first-out order. Sending a notification enqueues the request. The request processor then processes items from the queue at a later time. Requests can be handled directly or routed to interested parties. Decoupling the sender from the receiver both statically and in time."

A decent definition, though dense, let's break it down.

Breaking It Down

"Stores a series of requests in first-in, first-out order."

- This is the Queue part of our pattern
- When many requests are made they are handled in the order they were made
- This is a relatively common data structure, but not everyone has taken GDAPS II or DSA yet

Breaking It Down

"Sending a notification enqueues the request. The request processor then processes items from the queue at a later time."

- This means that while the request are handled in order, the time they are handled is undetermined.
- This lets us buffer the event and not stop everything dead in its tracks when the event fires, which is what we're trying to avoid by using this pattern

Breaking It Down

"Requests can be handled directly or routed to interested parties. Decoupling the sender from the receiver both statically and in time."

- Ignore or utilize the events anywhere in our program.
- The object that sends the request and the object that acts on the event don't have to be entangled to work.

Implementation

Way too time consumingly complicated to address here, this is a sufficiently advanced pattern, if you're curious on its implementation, I'd encourage you to read up on it here.

Service Locator

"A service class defines an abstract interface to a set of operations. A concrete provider implements this interface. A service locator provides access to the service by finding an appropriate provider, hiding both the provider's concrete type and the process used to locate it."

OOF, THAT'S BAD.

Better definition:

"Provide access to functions without making an object that has the functions."

Implementation

The service

```
class Audio
{
public:
  virtual ~Audio() {}
  virtual void playSound(int soundID) = 0;
  virtual void stopSound(int soundID) = 0;
  virtual void stopAllSounds() = 0;
};
```

In the code

```
ConsoleAudio *audio = new ConsoleAudio();
Locator::provide(audio);

Audio *audio = Locator::getAudio();
audio->playSound(VERY_LOUD_BANG);
```

The service provider

```
class ConsoleAudio : public Audio
{
public:
  virtual void playSound(int soundID)
  {
    // Play sound using console audio api...
  }

  virtual void stopSound(int soundID)
  {
    // Stop sound using console audio api...
  }

  virtual void stopAllSounds()
  {
    // Stop all sounds using console audio api...
  }
};
```

The locator

```
class Locator
{
public:
  static Audio* getAudio() { return service_; }

  static void provide(Audio* service)
  {
    service_ = service;
  }

private:
  static Audio* service_;
};
```

Optimization Patterns

Object Pool

Spatial Partition

Object Pool

"A class that maintains a collection of reusable objects."

- Maintain a fixed pool instead of allocating and deleting objects individually
- Reduce fragmentation in C++
- Packs a bunch of objects together in an easy to manage area in the code

Why use that???

When we have a large collection of the same object that consistently gets used over and over.

- Particle Effects
- Enemies
- Notes in a rhythm game

(Brief) Implementation

```
class ParticlePool
{
public:
  void create(double x, double y,
              double xVel, double yVel, int lifetime);

  void animate()
  {
    for (int i = 0; i < POOL_SIZE; i++)
    {
      particles_[i].animate();
    }
  }

private:
  static const int POOL_SIZE = 100;
  Particle particles_[POOL_SIZE];
};
```

```
void ParticlePool::create(double x, double y,
                          double xVel, double yVel,
                          int lifetime)
{
  // Find an available particle.
  for (int i = 0; i < POOL_SIZE; i++)
  {
    if (!particles_[i].inUse())
    {
      particles_[i].init(x, y, xVel, yVel, lifetime);
      return;
    }
  }
}
```

Spatial Partition

"Store objects in a data structure that organizes the objects by their positions."

- Stores locality of objects relative to each other
- Update the data structure when objects transform positions
- Efficiently query for object near x location

Why do we want to do this?

- Collision Detection
  - Every object has to check every other object
  - If we have X number of objects, becomes a runtime bog to check
  - Dividing our objects into spaces to collide in minimizes the operation
- Audio SFX
  - Check how close they are to the player
  - Scale volume or not play at all

(Brief) Implementation

```
Unit::Unit(Grid* grid, double x, double y)
: grid_(grid),
  x_(x),
  y_(y),
  prev_(NULL),
  next_(NULL)
{
  grid_->add(this);
}
```

```
void Grid::add(Unit* unit)
{
  // Determine which grid cell it's in.
  int cellX = (int)(unit->x_ / Grid::CELL_SIZE);
  int cellY = (int)(unit->y_ / Grid::CELL_SIZE);

  // Add to the front of list for the cell it's in.
  unit->prev_ = NULL;
  unit->next_ = cells_[cellX][cellY];
  cells_[cellX][cellY] = unit;

  if (unit->next_ != NULL)
  {
    unit->next_->prev_ = unit;
  }
}
```

Questions?

Homework

Read the book / website / resources I stole all the information from this presentation from:

Game Development Patterns

Gang of Four Design Pattern